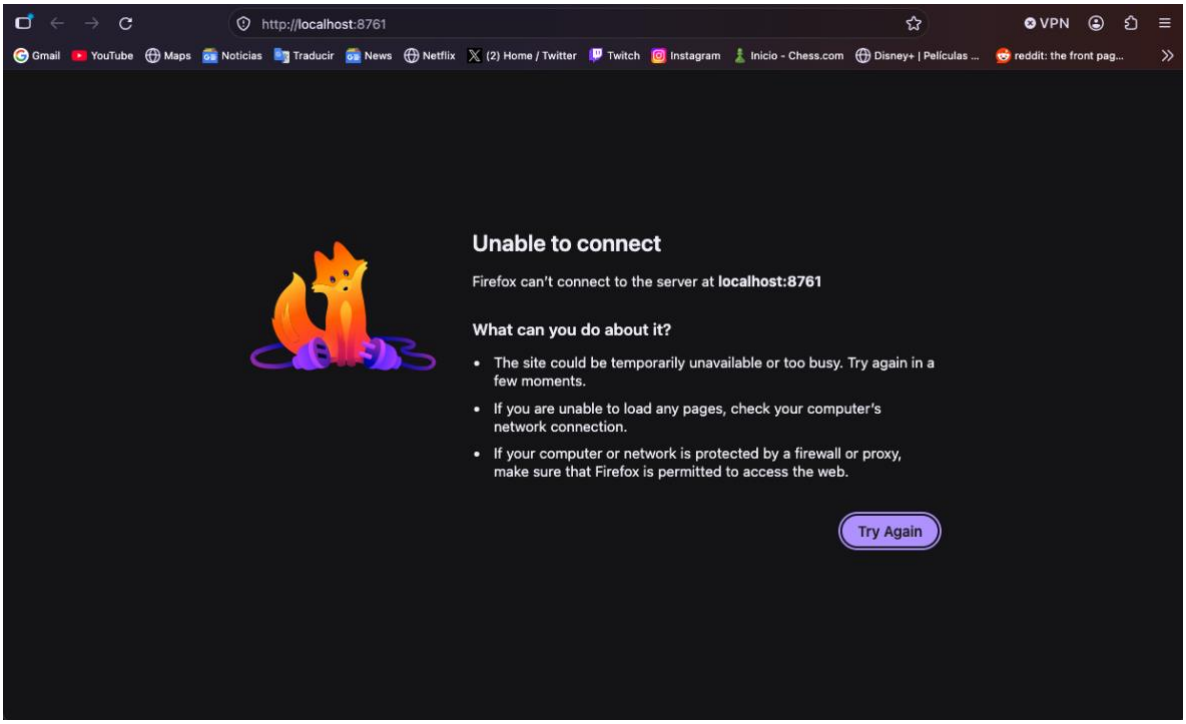
Paso 1. La historia

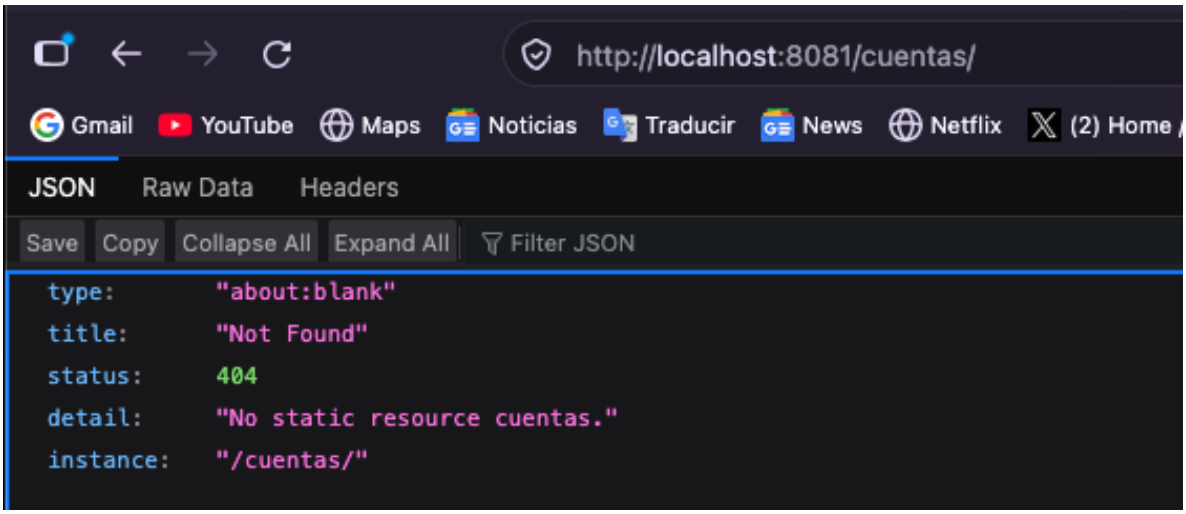
Cuando estábamos trabajando con un monolito y hacíamos una operación se llamaba a un método para realizarlo. Si la operación se hacía de manera exitosa entonces se actualizaba la base de datos y si fallaba se podía hacer un rollback sin problemas. Ahora que se está trabajando con llamadas HTTP cada cliente debe hacer peticiones para poder realizar operaciones. La app se divide en 5 microservicios donde cada uno puede fallar y hacer que la misma operación en un servicio haya sido exitosa, mientras que e otro falló. Mantener consistente la base de datos se vuelve un reto.

Paso 2. Directorio

Al empezar la práctica a las 12:30pm no se tiene un directorio activo y el puerto 8761 no desplegaba nada.



De la misma forma, el puerto 8081 que expone al servicio de cuentas despliega un error 404



Para poderlo encender modificamos el archivo DiscoveryApplication.java y agregamos EnableEurekaServer en las librerías y en el código.

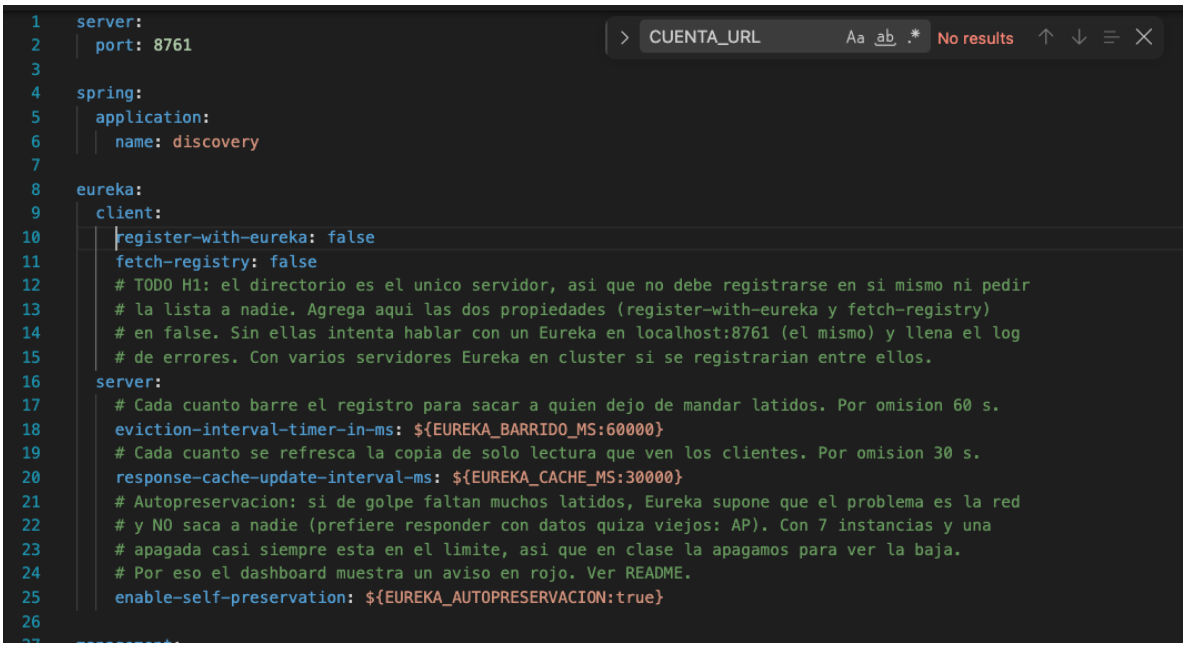
```java
package mx.mexibanco;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

/**
 * El directorio de Mexi Banco v06. Cada servicio se da de alta aqui con su nombre logico
 * (cuenta, movimiento, ...) y su IP, y manda un latido cada tanto. Quien quiere hablar con
 * "cuenta" pregunta aqui que instancias hay, en vez de tener escrita una URL.
 *
 * Dashboard: http://localhost:8761
 */
@SpringBootApplication
@EnableEurekaServer
// TODO H1: esta aplicacion todavia no es un directorio. Agrega la anotacion que la convierte en
// servidor Eureka (paquete org.springframework.cloud.netflix.eureka.server; la dependencia ya
// esta en el pom). Sin ella la aplicacion ni siquiera arranca: el cliente de Eureka que trae el
// starter busca un servidor que no existe y truena con NullPointerException en EurekaRegistration.
public class DiscoveryApplication {

    public static void main(String[] args) {
        SpringApplication.run(DiscoveryApplication.class, args);
    }
}
```

Luego modificamos el archivo application.yaml del Discovery para que el directorio que levantemos no se quiera anotar en sí mismo con las líneas registry-with-eureka: false y fetch-registry: false.

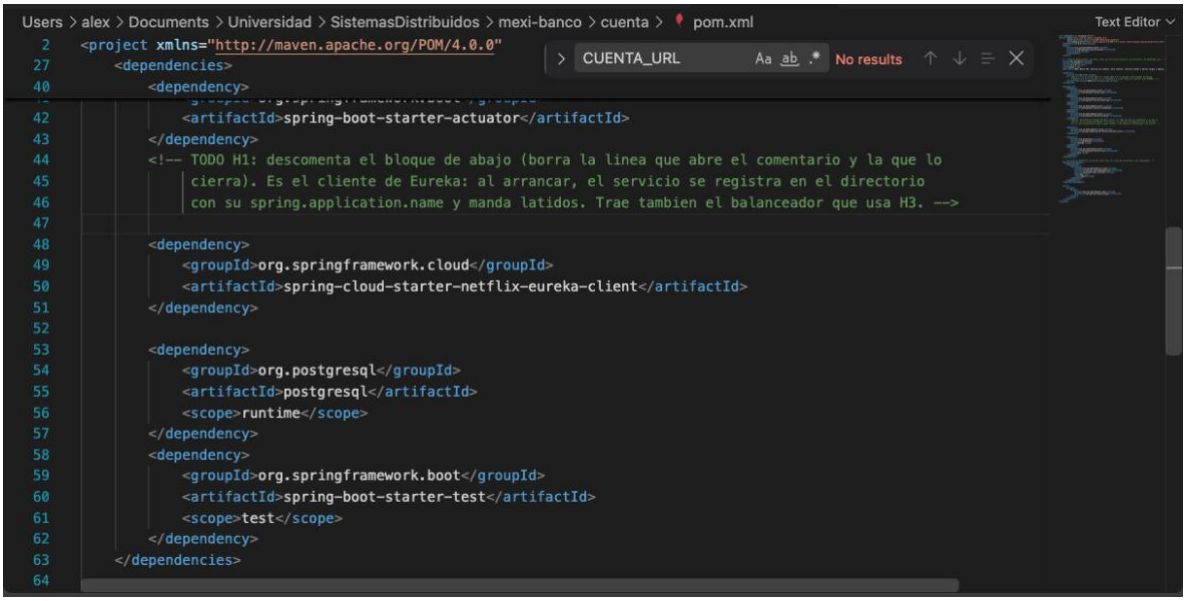



```yaml
server:
  port: 8761

spring:
  application:
    name: discovery

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
    # TODO H1: el directorio es el unico servidor, asi que no debe registrarse en si mismo ni pedir
    # la lista a nadie. Agrega aqui las dos propiedades (register-with-eureka y fetch-registry)
    # en false. Sin ellas intenta hablar con un Eureka en localhost:8761 (el mismo) y llena el log
    # de errores. Con varios servidores Eureka en cluster si se registrarian entre ellos.
  server:
    # Cada cuanto barre el registro para sacar a quien dejo de mandar latidos. Por omision 60 s.
    eviction-interval-timer-in-ms: ${EUREKA_BARRIDO_MS:60000}
    # Cada cuanto se refresca la copia de solo lectura que ven los clientes. Por omision 30 s.
    response-cache-update-interval-ms: ${EUREKA_CACHE_MS:30000}
    # Autopreservacion: si de golpe faltan muchos latidos, Eureka supone que el problema es la red
    # y NO saca a nadie (prefiere responder con datos quiza viejos: AP). Con 7 instancias y una
    # apagada casi siempre esta en el limite, asi que en clase la apagamos para ver la baja.
    # Por eso el dashboard muestra un aviso en rojo. Ver README.
    enable-self-preservation: ${EUREKA_AUTOPRESERVACION:true}
```

No obstante, los demás servicios sí se deben anotar en el directorio, por lo que descomentamos de todos los archivos pom.xml las líneas comentadas.

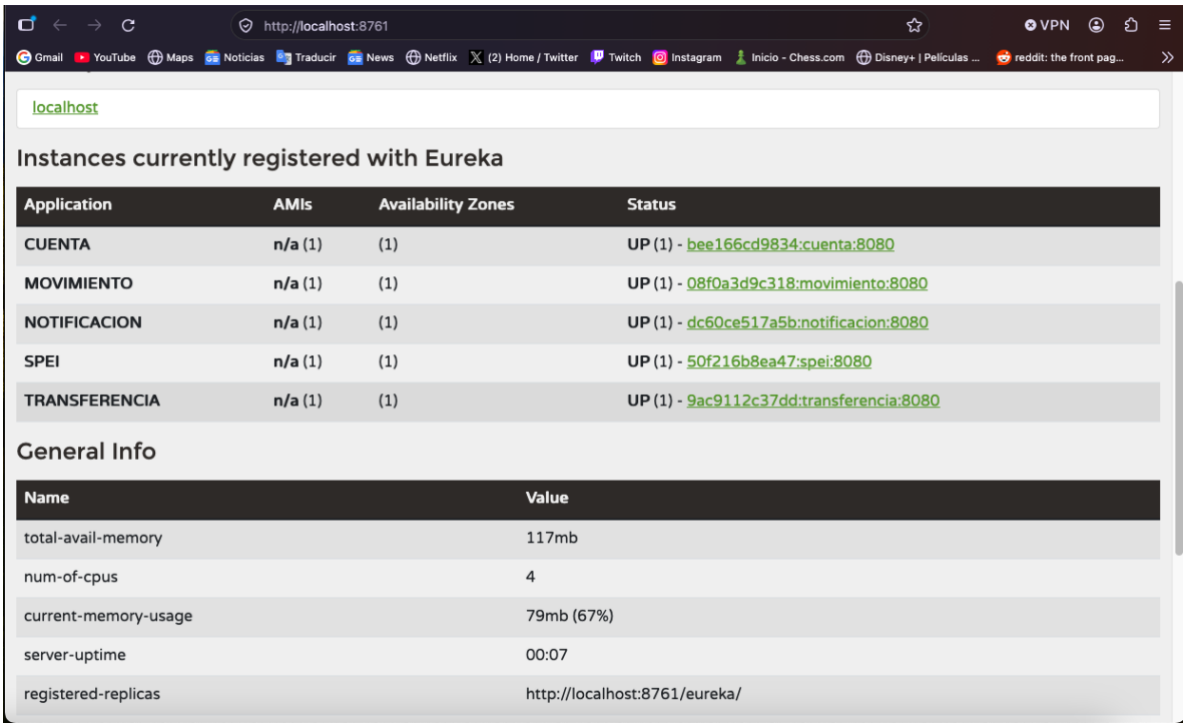



```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
    <dependencies>
        <dependency>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
        <!-- TODO H1: descomenta el bloque de abajo (borra la linea que abre el comentario y la que lo
             cierra). Es el cliente de Eureka: al arrancar, el servicio se registra en el directorio
             con su spring.application.name y manda latidos. Trae tambien el balanceador que usa H3. -->

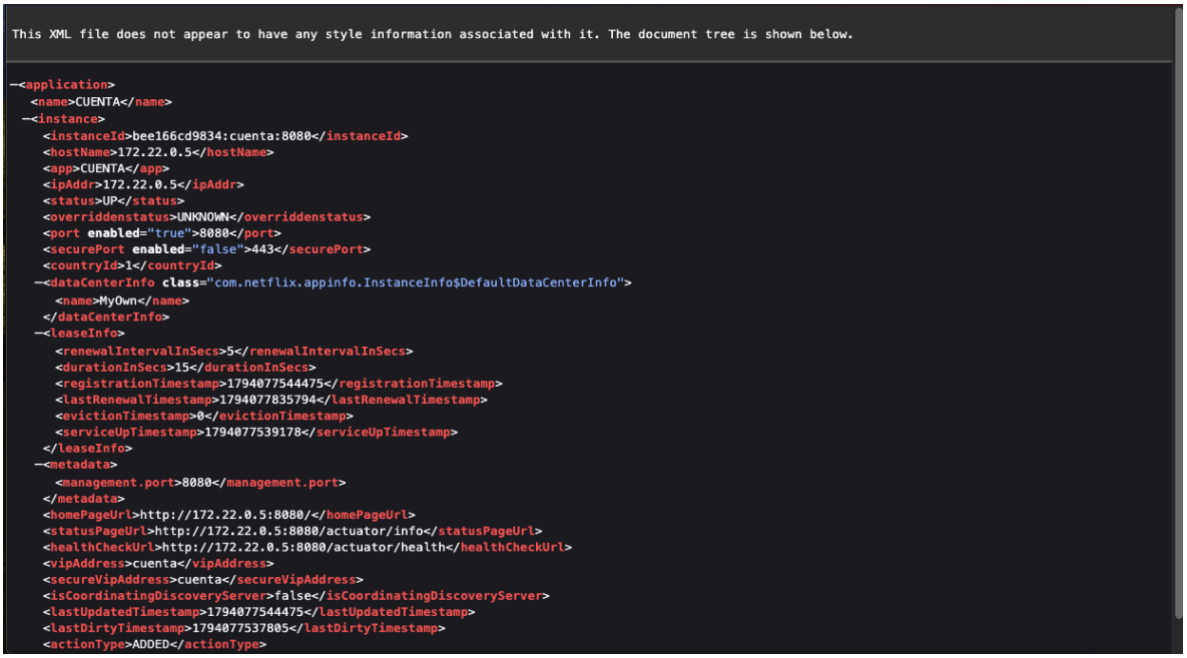
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>

        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

Con esto los servicios van a anotarse en el directorio y mandar heartbeats a través de Eureka. Con esto y descomentando las líneas del servicio discovery del archivo .yml de la app podemos cargar el servicio de Eureka.



Y el puerto 8081 ya despliega la información de cuentas.



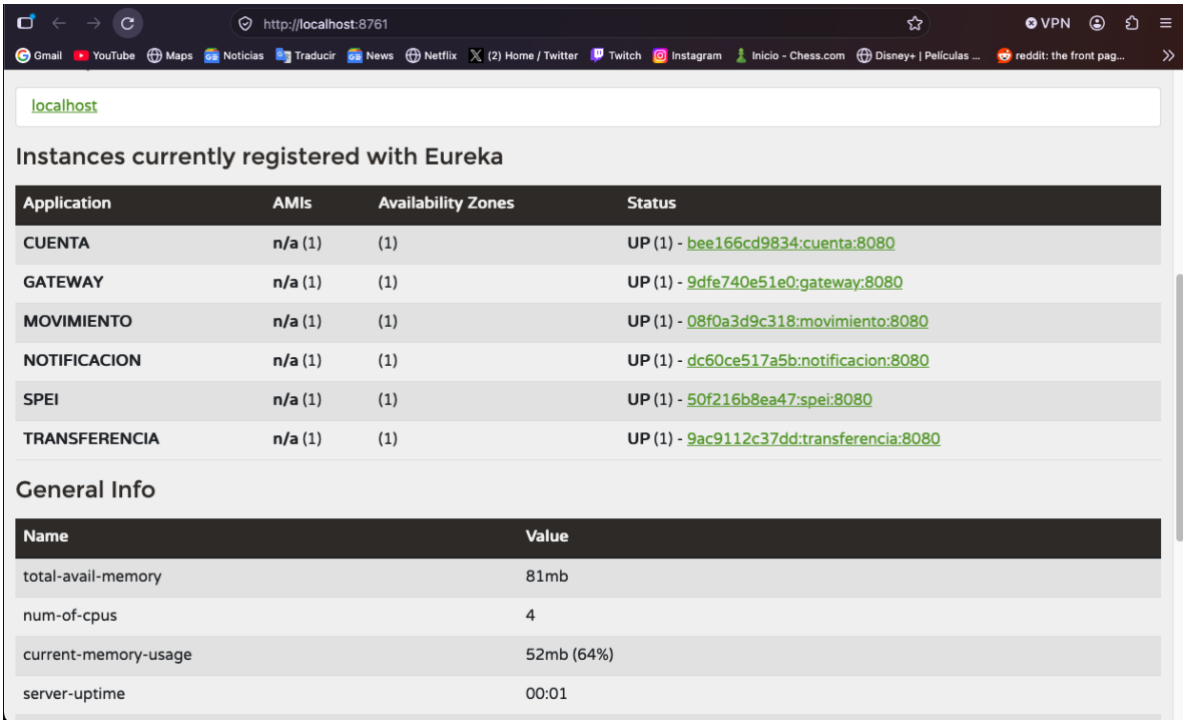
Pregunta Piensa: en el compose, transferencia sigue llamando a CUENTA_URL: http://cuenta:8080, una dirección escrita a mano. ¿Alguien está usando el directorio para encontrar a cuenta? No, en este momento todos los servicios que llaman a cuenta tienen la dirección escrita manualmente (transferencia y spei).

Parte 3. Gateway

Antes de levantar el Gateway intentamos mandar una solicitud a transferencia directamente con un comando, pero nos regresa un error 400 porque registrar un movimiento es una acción que solo se debe hacer entre servicios.

```
(base) alex@Apples-MacBook-Pro mexi-banco % curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sat, 07 Nov 2026 19:01:52 GMT
Connection: close
```

Para ello ya se tiene que levantar el gateway para que se encargue de canalizar las peticiones.



Con el Gateway levantado ya se pueden realizar operaciones. La primera quiere consultar una cuenta que no existe.

```
(base) alex@Apples-MacBook-Pro mexi-banco % curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: bee166cd9834
Content-Type: application/problem+json
Date: Sat, 07 Nov 2026 19:13:20 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}%
```

La segunda intenta registrar un POST a movimientos, pero a ese servicio solo se le tiene Method=GET.

```
(base) alex@Apples-MacBook-Pro mexi-banco % curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-11-07T19:14:14.997+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"cce26e31-28"}%
```

Posteriormente se crea una cuenta de prueba con el titular S07 y un saldo inicial de 0. La petición devuelve 201 porque sí se efectuó la operación.

```
(base) alex@Apples-MacBook-Pro mexi-banco % LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
[  -d "{\"clabe\":\"$LAB_CLABE\",\"titular\":\"S07\",\"saldoInicial\":0}"
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: bee166cd9834
Content-Type: application/json
Date: Sat, 07 Nov 2026 19:15:35 GMT

{"id":7,"clabe":"S071794078935","titular":"S07","saldo":0}%
```

Luego se consultó la cuenta recién creada y nos devolvió la información con el mensaje OK.

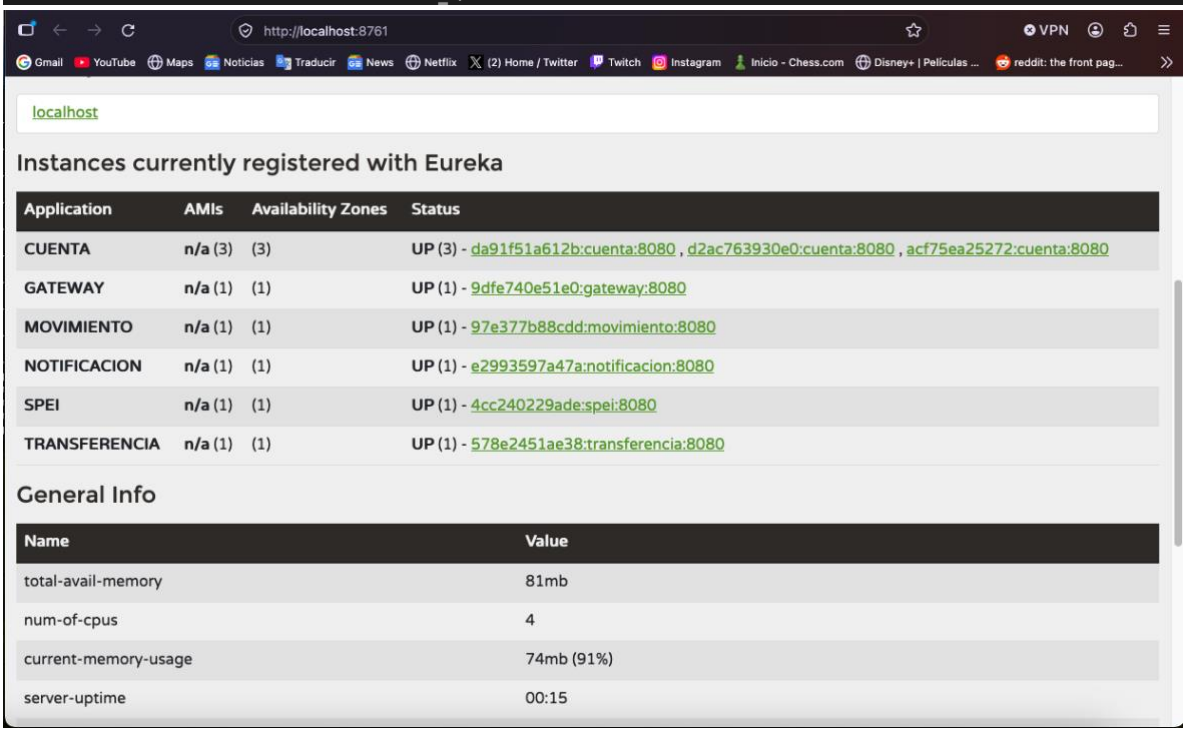
```
(base) alex@Apples-MacBook-Pro mexi-banco % URL="http://localhost:8080/api/cuentas/$LAB_CLABE"
(base) alex@Apples-MacBook-Pro mexi-banco % curl -i "$URL"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: bee166cd9834
Content-Type: application/json
Date: Sat, 07 Nov 2026 19:16:22 GMT

{"id":7,"clabe":"S071794078935","titular":"S07","saldo":0.00}%
(base) alex@Apples-MacBook-Pro mexi-banco % echo "$URL"
http://localhost:8080/api/cuentas/S071794078935
```

Paso 4. Copias

Después de modificar el archivo de docker-compose para que cuentas tuviera 3 replicas dentro del deploy volvemos a construir cuentas y se actualiza el tablero de Eureka con las 3 instancias de cuenta.

```
(base) alex@Apples-MacBook-Pro mexi-banco % docker ps --filter name=cuenta --format '{{.ID}}  {{.Names}}  {{.Ports}}'
da91f51a612b  mexi-banco-v06-cuenta-1  8080/tcp
d2ac763930e0  mexi-banco-v06-cuenta-3  8080/tcp
acf75ea25272  mexi-banco-v06-cuenta-2  8080/tcp
```



Luego probamos al Gateway para ver cómo está repartiendo las peticiones con un for que hace llamadas a las instancias de cuentas.

```
(base) alex@Apples-MacBook-Pro mexi-banco % for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: da91f51a612b
X-Instancia: acf75ea25272
X-Instancia: d2ac763930e0
X-Instancia: da91f51a612b
X-Instancia: acf75ea25272
X-Instancia: d2ac763930e0
X-Instancia: da91f51a612b
X-Instancia: acf75ea25272
X-Instancia: d2ac763930e0
```

```
cf75ea25272
cuenta-1  | 2026-11-07T19:28:40.463Z  INFO 1 --- [cuenta] [nio-8080-exec-3] mx.m
exibanco.cuenta.CabeceraInstancia    : GET /cuentas/S071794078935 atendida por d
a91f51a612b
cuenta-3  | 2026-11-07T19:28:40.527Z  INFO 1 --- [cuenta] [nio-8080-exec-2] mx.m
exibanco.cuenta.CabeceraInstancia    : GET /cuentas/S071794078935 atendida por d
2ac763930e0
cuenta-2  | 2026-11-07T19:28:40.585Z  INFO 1 --- [cuenta] [nio-8080-exec-3] mx.m
exibanco.cuenta.CabeceraInstancia    : GET /cuentas/S071794078935 atendida por a
cf75ea25272
cuenta-1  | 2026-11-07T19:28:40.657Z  INFO 1 --- [cuenta] [nio-8080-exec-2] mx.m
exibanco.cuenta.CabeceraInstancia    : GET /cuentas/S071794078935 atendida por d
a91f51a612b
cuenta-3  | 2026-11-07T19:28:40.735Z  INFO 1 --- [cuenta] [nio-8080-exec-3] mx.m
exibanco.cuenta.CabeceraInstancia    : GET /cuentas/S071794078935 atendida por d
2ac763930e0
cuenta-2  | 2026-11-07T19:28:40.786Z  INFO 1 --- [cuenta] [nio-8080-exec-4] mx.m
exibanco.cuenta.CabeceraInstancia    : GET /cuentas/S071794078935 atendida por a
cf75ea25272
cuenta-1  | 2026-11-07T19:28:40.841Z  INFO 1 --- [cuenta] [nio-8080-exec-5] mx.m
exibanco.cuenta.CabeceraInstancia    : GET /cuentas/S071794078935 atendida por d
a91f51a612b
cuenta-3  | 2026-11-07T19:28:40.898Z  INFO 1 --- [cuenta] [nio-8080-exec-4] mx.m
exibanco.cuenta.CabeceraInstancia    : GET /cuentas/S071794078935 atendida por d
```

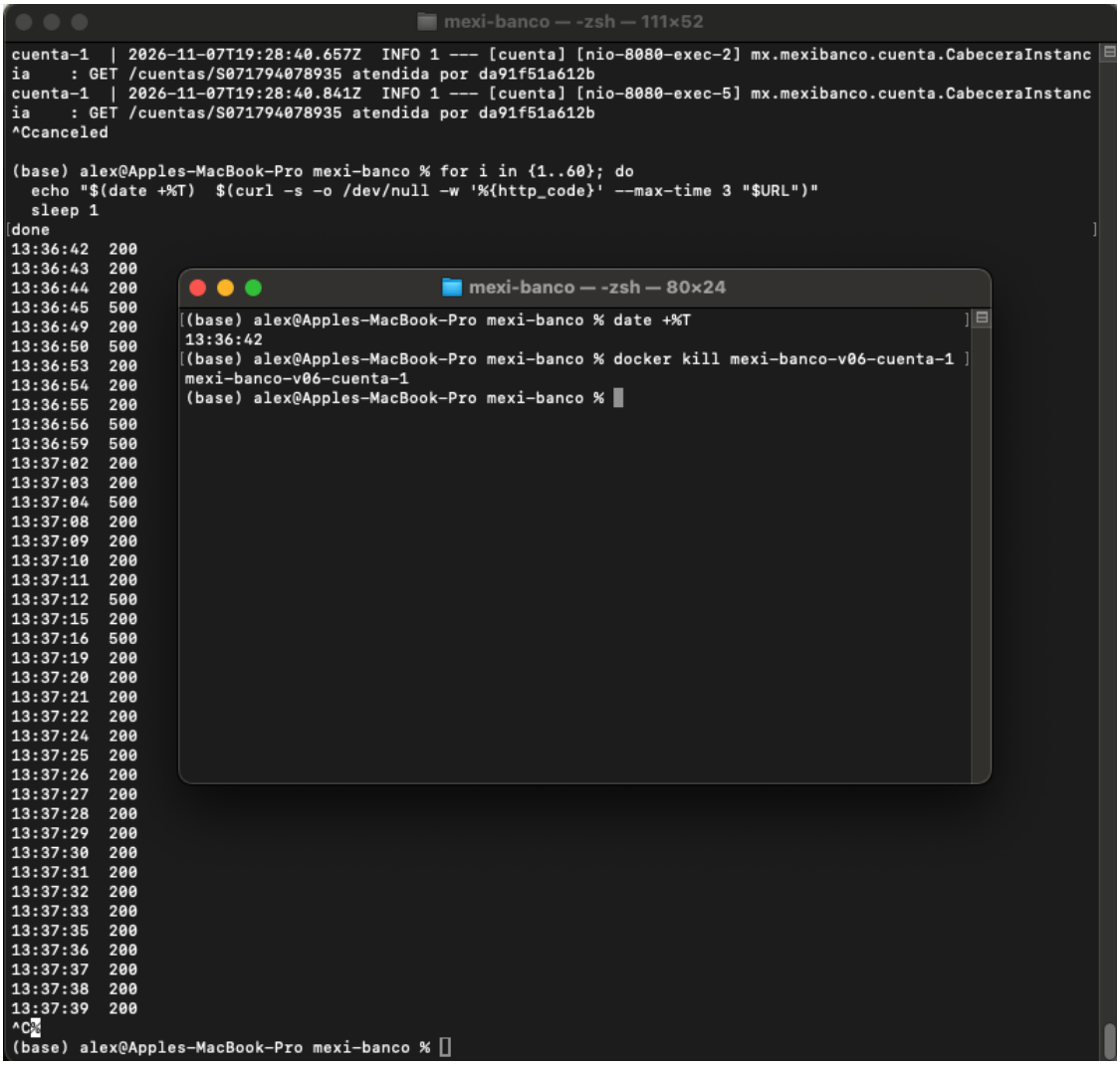
Pregunta Piensa: Esto prueba que el gateway reparte. ¿Y cuando transferencia llama a cuenta por dentro, también reparte? En este caso no reparte porque está llamando al puerto 8080 de cuenta. El DNS devuelve las 3 réplicas y se decide usar alguna de las 3. Esta réplica siempre va a estarle respondiendo a transferencia.

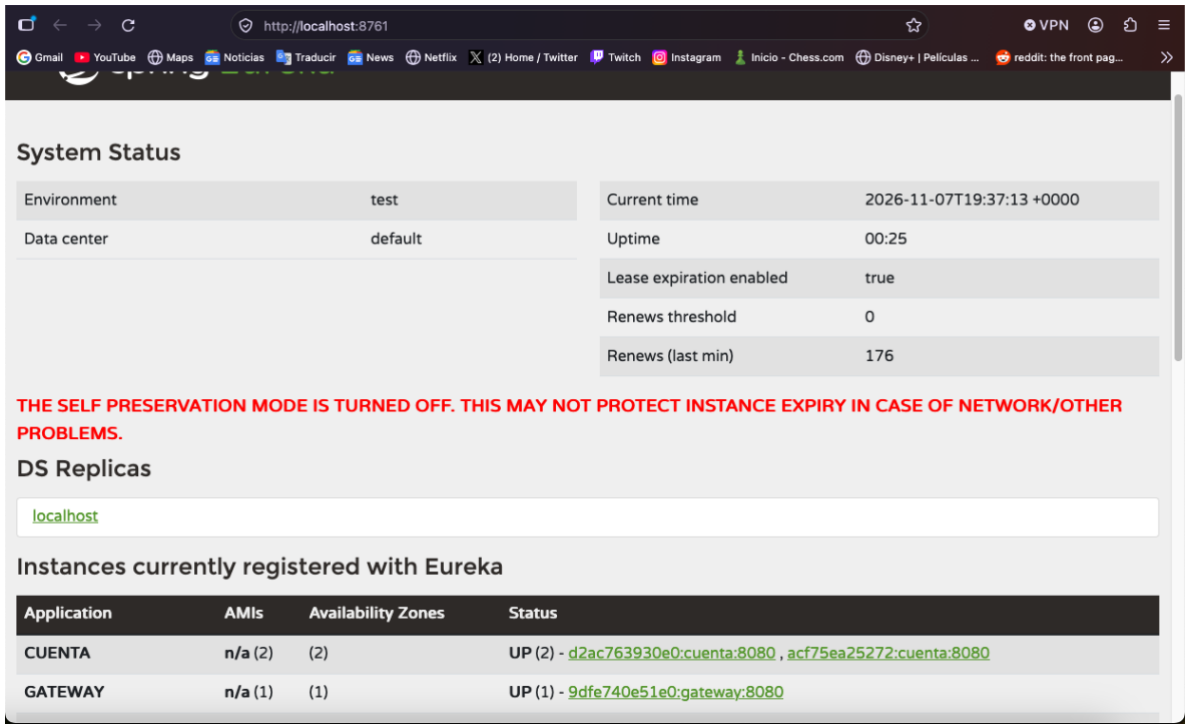
Paso 5. Caída

Mi predicción es que van a fallar las consultas que se dirijan a la instancia caída en el periodo donde el gateway todavía no sabe que está muerta. En el momento que gateway intente escuchar su heartbeat y no lo escuché dejará de mandar peticiones a la instancia y ya no habrá errores.

Al realizar la prueba de la copia caída recaudé los siguiente:

El for comenzó a las 13:36:42 y duró hasta las 13:37:39 porque lo aborté con CTRL C. En ese tiempo hubo 7 errores (500) de las 40 peticiones. En Eureka se actualizó el número de replicas a 2 a las 13:37:13, mientras que la muerte sucedió a las 13:36:42. Es decir, Eureka tardó aproximadamente 31 segundos en actualizarse y se dejaron de enviar peticiones a la réplica muerta después de 34 segundos.





Cierre

1. El directorio resuelve dónde está cada servicio vivo. Monitorea los latidos de cada servicio para determinar su estado y cuando le preguntan la ubicación de un servicio responde con las instancias vivas.
2. El gateway resuelve el tráfico de peticiones del exterior a la app. Reparte las peticiones a las rutas de los servicios vivos que guarda el directorio.
3. El balanceador resuelve a quién mandar cada petición. Es quien decide a que réplica enviar una petición

Tarea

```
(base) alex@Apples-MacBook-Pro mexi-banco % ./demo-v06.sh

=== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ===
-> listo en 1 s. Lo que sabe el directorio (http://localhost:8761/eureka/apps):
   CUENTA         3 instancia(s)
   GATEWAY        1 instancia(s)
   MOVIMIENTO     1 instancia(s)
   NOTIFICACION   1 instancia(s)
   SPEI           1 instancia(s)
   TRANSFERENCIA  1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

=== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009408
383001","titular":"Ana","saldoInicial":1000}'
{
  "id": 8,
  "clabe": "002180009408383001",
  "titular": "Ana",
  "saldo": 1000
}
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

=== 2. Abrir la cuenta de Beto con 500 (misma puerta) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009408
383002","titular":"Beto","saldoInicial":500}'
{
  "id": 9,
  "clabe": "002180009408383002",
  "titular": "Beto",
  "saldo": 500
}
[HTTP 201]
```

```
=== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":
"002180009408383001","claveDestino":"002180009408383002","monto":200}'
{
  "id": 3,
  "claveOrigen": "002180009408383001",
  "claveDestino": "002180009408383002",
  "monto": 200,
  "fecha": "2026-11-07T20:37:17.918104146Z"
}
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
   cuenta-3  GET /cuentas/002180009408383001 atendida por d2ac763930e0
   cuenta-1  GET /cuentas/002180009408383002 atendida por da91f51a612b
   cuenta-2  POST /cuentas/002180009408383001/cargos atendida por acf75ea25272
   cuenta-3  POST /cuentas/002180009408383002/abonos atendida por d2ac763930e0
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

=== 4. Consultar esa transferencia ===
$ curl http://localhost:8080/api/transferencias/3
{
  "id": 3,
  "claveOrigen": "002180009408383001",
  "claveDestino": "002180009408383002",
  "monto": 200,
  "fecha": "2026-11-07T20:37:17.918104Z"
}
[HTTP 200]

=== 5. Saldos: el dinero no se crea ni se pierde ===
$ curl http://localhost:8080/api/cuentas/002180009408383001
{
  "id": 8,
  "clabe": "002180009408383001",
  "titular": "Ana",
  "saldo": 800
}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009408383002
{
  "id": 9,
  "clabe": "002180009408383002",
  "titular": "Beto",
  "saldo": 700
}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700
```

```
=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===
$ curl http://localhost:8080/api/movimientos?clabe=002180009408383001
[
  {
    "id": 13,
    "clabeCuenta": "002180009408383001",
    "tipo": "TRANSFERENCIA_ENVIADA",
    "monto": -200,
    "saldoResultante": 800,
    "fecha": "2026-11-07T20:37:17.832910Z",
    "referencia": "a 002180009408383002"
  },
  {
    "id": 11,
    "clabeCuenta": "002180009408383001",
    "tipo": "DEPOSITO",
    "monto": 1000,
    "saldoResultante": 1000,
    "fecha": "2026-11-07T20:37:13.212627Z",
    "referencia": "saldo inicial"
  }
]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clabe=002180009408383002
[
  {
    "id": 8,
    "clabeCuenta": "002180009408383002",
    "mensaje": "TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00",
    "fecha": "2026-11-07T20:37:19.584381Z"
  }
]
[HTTP 200]
```

```
=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-dem
o-94083830' -d '{"claveOrigen":"002180009408383001","bancoDestino":"BANCO-X","claveDestino":"012180000000000009
","monto":50}'
{
  "id": 5,
  "idempotencyKey": "spei-demo-94083830",
  "claveOrigen": "002180009408383001",
  "bancoDestino": "BANCO-X",
  "claveDestino": "012180000000000009",
  "monto": 50,
  "estado": "ENVIADO"
}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-dem
o-94083830' -d '{"claveOrigen":"002180009408383001","bancoDestino":"BANCO-X","claveDestino":"012180000000000009
","monto":50}'
{
  "id": 5,
  "idempotencyKey": "spei-demo-94083830",
  "claveOrigen": "002180009408383001",
  "bancoDestino": "BANCO-X",
  "claveDestino": "012180000000000009",
  "monto": 50,
  "estado": "ENVIADO"
}
[HTTP 201]
-> mismo id (5 -> 5): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ===
   consulta 1  X-Instancia: acf75ea25272  (mexi-banco-v06-cuenta-2)
   consulta 2  X-Instancia: d2ac763930e0  (mexi-banco-v06-cuenta-3)
   consulta 3  X-Instancia: acf75ea25272  (mexi-banco-v06-cuenta-2)
   consulta 4  X-Instancia: da91f51a612b  (mexi-banco-v06-cuenta-1)
   consulta 5  X-Instancia: d2ac763930e0  (mexi-banco-v06-cuenta-3)
   consulta 6  X-Instancia: acf75ea25272  (mexi-banco-v06-cuenta-2)
   consulta 7  X-Instancia: da91f51a612b  (mexi-banco-v06-cuenta-1)
   consulta 8  X-Instancia: d2ac763930e0  (mexi-banco-v06-cuenta-3)
   consulta 9  X-Instancia: acf75ea25272  (mexi-banco-v06-cuenta-2)
-> 3 replicas distintas en 9 consultas; 0 veces se repitio la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080
```

```
=== 9. Los endpoints internos no tienen puerta ===
$ curl -X POST http://localhost:8080/api/cuentas/002180009408383001/abonos -H 'Content-Type: application/json'
-d '{"monto":1000000}'
{
  "timestamp": "2026-11-07T20:37:29.126+00:00",
  "path": "/api/cuentas/002180009408383001/abonos",
  "status": 404,
  "error": "Not Found",
  "requestId": "139ab85e-1091"
}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas ade
lante)

=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":
"002180009408383001","claveDestino":"002180009408383099","monto":10}'
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009408383099",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":
"002180009408383001","claveDestino":"002180009408383002","monto":99999}'
{
  "type": "about:blank",
  "title": "Unprocessable Entity",
  "status": 422,
  "detail": "Saldo insuficiente en la CLABE 002180009408383001",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 422]

=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caida: ./demo-v06.sh --latido
```

```
=== 12. El latido: matar una replica de cuenta sin que se despida ===
-> en el directorio hay 3 instancias de CUENTA; la victima es acf75ea25272 (mexi-banco-v06-cuenta-2)
-> docker kill = se va la luz: el proceso no alcanza a darse de baja. Solo dejan de llegar sus latidos.
-> (docker stop seria un apagado ordenado: Spring la marca DOWN al instante y nadie le manda trafico)
   t=  0s  CUENTA en el directorio: 3   por la puerta: 1 bien, 0 con error
   t=  5s  CUENTA en el directorio: 3   por la puerta: 3 bien, 1 con error
   t= 25s  CUENTA en el directorio: 3   por la puerta: 11 bien, 5 con error
   t= 30s  CUENTA en el directorio: 3   por la puerta: 13 bien, 6 con error
   t= 35s  CUENTA en el directorio: 3   por la puerta: 15 bien, 6 con error
-> el directorio saco a la replica muerta a los 38 s
-> mientras tanto, por la puerta: 15 respuestas bien y 7 con error (el ultimo error a los 36 s)
-> los errores son el gateway mandando peticiones a una replica que ya no existe: el registro es eventualmente
consistente
   Lo que sabe ahora el directorio:
   CUENTA         2 instancia(s)
   GATEWAY        1 instancia(s)
   MOVIMIENTO     1 instancia(s)
   NOTIFICACION   1 instancia(s)
   SPEI           1 instancia(s)
   TRANSFERENCIA  1 instancia(s)

=== 13. Volver a prender la replica y ver cuanto tarda en reaparecer ===
-> mexi-banco-v06-cuenta-2 aparece otra vez en el directorio a los 33 s de docker start (arranque de la JVM + r
egistro)
-> y el gateway le vuelve a mandar peticiones a los 39 s

Fin. Con EUREKA_EXPIRA_S=15 EUREKA_LATIDO_S=5 EUREKA_CONSULTA_S=5 BALANCEADOR_CACHE=5s (Spring Cloud trae latid
o 30, expiracion 90, consulta 30 y cache 35s).
(base) alex@Apples-MacBook-Pro mexi-banco % docker compose logs cuenta | grep -E 'cargos|abonos'
cuenta-3  | 2026-11-07T20:37:17.299Z  INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstanc
ia     : POST /cuentas/002180009408383002/abonos atendida por d2ac763930e0
cuenta-3  | 2026-11-07T20:37:29.457Z  INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstanc
ia     : POST /cuentas/002180009408383001/cargos atendida por d2ac763930e0
cuenta-3  | 2026-11-07T20:41:49.093Z  INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstanc
ia     : POST /cuentas/002180009408410701/cargos atendida por d2ac763930e0
cuenta-1  | 2026-11-07T20:37:25.788Z  INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstanc
ia     : POST /cuentas/002180009408383001/cargos atendida por da91f51a612b
cuenta-1  | 2026-11-07T20:41:52.136Z  INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstanc
ia     : POST /cuentas/002180009408410701/cargos atendida por da91f51a612b
cuenta-1  | 2026-11-07T20:41:55.757Z  INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstanc
ia     : POST /cuentas/002180009408410701/cargos atendida por da91f51a612b
cuenta-2  | 2026-11-07T20:37:17.183Z  INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstanc
ia     : POST /cuentas/002180009408383001/cargos atendida por acf75ea25272
cuenta-2  | 2026-11-07T20:41:49.169Z  INFO 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CabeceraInstanc
ia     : POST /cuentas/002180009408410702/abonos atendida por acf75ea25272
```